

Delibera — Upgrade to .NET 10 and C# 15

This document describes the changes made while migrating the **Delibera** project to the **.NET 10** platform and the **C# 15 (preview)** language, the performance optimizations, and the new examples of the **Operator** role working with MCP tools (browser and Marp presentation generation).

All changes were made in the `feature/operator-mcp-role` branch. The project builds with `dotnet build` without warnings or errors.

Table of Contents

- [1. Target Platform and Language Version](#)
- [2. Dependency Updates](#)
- [3. Performance Optimizations \(high performance\)](#)
- [4. Operator + MCP Tools Examples](#)
- [5. Build and Run](#)
- [6. Summary of Changed Files](#)

1. Target Platform and Language Version

Both projects were migrated to a single stack:

Property	Before	After
<code>TargetFramework</code>	<code>net10.0</code>	<code>net10.0</code>
<code>LangVersion</code>	<code>14.0</code>	<code>preview (C# 15)</code>
<code>Nullable</code>	<code>enable</code>	<code>enable</code>
<code>ImplicitUsings</code>	<code>enable</code>	<code>enable</code>

The changes were made in the files:

- `src/Delibera.Core/Delibera.Core.csproj`
- `src/Delibera.ConsoleApp/Delibera.ConsoleApp.csproj`

Important note about “C# 15”

At the time of the upgrade, the GA release **.NET SDK 10.0.301** is installed, in which the Roslyn compiler supports language versions up to and including **14.0**, while the next one (the future C# 15) is available **only** through the `preview` value. Specifying `<LangVersion>15.0</LangVersion>` directly results in a compilation error:

```
error CS1617: Invalid option '15.0' for /langversion
```

Therefore `<LangVersion>preview</LangVersion>` is used — this enables the maximum available set of language features of the next version (C# 15). When an SDK with official `15.0` support is released, the value can be replaced with `15.0` without any other changes to the code.

2. Dependency Updates

All NuGet packages were brought to versions compatible with .NET 10. Most packages were already on their current versions for .NET 10; `Microsoft.SourceLink.GitHub` was updated.

Delibera.Core

Package	Version	Purpose
Microsoft.Extensions.Configuration.Abstractions	10.0.9	Configuration (DI)
Microsoft.Extensions.Configuration.Json	10.0.9	Reading appsettings.json
Microsoft.Extensions.DependencyInjection	10.0.9	DI container
Microsoft.Extensions.DependencyInjection.Abstractions	10.0.9	DI abstractions
Microsoft.Extensions.Options	10.0.9	Options pattern
Microsoft.Extensions.Options.ConfigurationExtensions	10.0.9	Binding Options to configuration
ModelContextProtocol	1.4.0	SDK for MCP servers (Operator role)
Npgsql	10.0.3	PostgreSQL / pgvector
OllamaSharp	5.4.25	Ollama LLM provider
Pgvector	0.3.2	Vector search in PostgreSQL
Qdrant.Client	1.18.1	Qdrant vector store
Microsoft.SourceLink.GitHub	10.0.300 (was 10.0.102)	Source Link for debugging

Delibera.ConsoleApp

Package	Version	Purpose
Mi-crosoft.Extensions.Configuration	10.0.9	Configuration
Mi-crosoft.Extensions.Configuration.Binder	10.0.9	Configuration binding
Mi-crosoft.Extensions.Configuration.Json	10.0.9	JSON configuration
Mi-crosoft.Extensions.Configuration.UserSecrets	10.0.9	User Secrets
Mi-crosoft.Extensions.DependencyInjection	10.0.9	DI container

The currency check was performed via `dotnet list package --outdated` and the NuGet flat-container

API — at the time of the upgrade there are no newer stable versions for the listed packages.

3. Performance Optimizations (high performance)

Targeted optimizations of the “hot” paths were made using `Span<T>` / `ReadOnlySpan<T>`, `SIMD` (`System.Numerics.Vector<T>`), and `ArrayPool<T>`. All optimizations preserve the previous behavior (verified numerically).

3.1. SIMD vectorization of cosine similarity

File: `src/Delibera.Core/Compression/SemanticCompressor.cs` → `CosineSimilarity`

Cosine similarity is the hottest loop during semantic compression and deduplication (it is called for every pair of sentences across all embedding dimensions, typically 768–1536). The implementation was rewritten:

- The signature was changed from `float[]` to `ReadOnlySpan<float>` — this is zero-copy and allows passing arrays, slices, and `stackalloc` buffers without allocations (calling code using `float[]` works without changes).
- The main loop is vectorized via `Vector<float>`: each iteration processes `Vector<float>.Count` elements (on modern x64/ARM64 — 8–16 values per cycle).
- The scalar “tail” handles the remainder and also serves as a fallback path when there is no hardware acceleration (`Vector.IsHardwareAccelerated == false`).

```

internal static double CosineSimilarity(ReadOnlySpan<float> a, ReadOnlySpan<float> b)
{
    if (a.Length != b.Length || a.IsEmpty) return 0;

    float dot = 0, magA = 0, magB = 0;
    var i = 0;

    if (Vector.IsHardwareAccelerated && a.Length >= Vector<float>.Count)
    {
        var dotAcc = Vector<float>.Zero;
        var magAAcc = Vector<float>.Zero;
        var magBAcc = Vector<float>.Zero;
        var width = Vector<float>.Count;

        for (; i <= a.Length - width; i += width)
        {
            var va = new Vector<float>(a.Slice(i, width));
            var vb = new Vector<float>(b.Slice(i, width));
            dotAcc += va * vb;
            magAAcc += va * va;
            magBAcc += vb * vb;
        }

        dot = Vector.Dot(dotAcc, Vector<float>.One);
        magA = Vector.Dot(magAAcc, Vector<float>.One);
        magB = Vector.Dot(magBAcc, Vector<float>.One);
    }

    for (; i < a.Length; i++) { dot += a[i]*b[i]; magA += a[i]*a[i]; magB += b[i]*b[i]
; }

    var denom = Math.Sqrt(magA) * Math.Sqrt(magB);
    return denom > 0 ? dot / denom : 0;
}

```

Correctness. A numerical comparison with the reference (scalar) implementation on lengths 1, 3, 7, 8, 16, 17, 100, 1536 yielded a maximum absolute error of $\approx 6 \cdot 10^{-8}$ (at the precision level of `float`); the result is identical.

3.2. Allocation-free token counting (`ReadOnlySpan<char>`)

File: `src/Delibera.Core/Compression/TokenCounter.cs`

- Added an `EstimateTokens(ReadOnlySpan<char> text)` overload — it allows estimating tokens for string slices without allocating substrings.
- The internal `CountWords` accepts `ReadOnlySpan<char>` and iterates over characters using a `ref struct` enumerator — without boxing and without copies.
- The existing `EstimateTokens(string?)` is preserved and now delegates to the span overload (via `text.AsSpan()`), so the public API is fully backward compatible.

3.3. Cache key hashing via `ArrayPool<T>` + `stackalloc`

File: `src/Delibera.Core/Compression/CompressionCache.cs` → `ComputeKey`

Previously, building the cache key created two extra heap allocations on every call: the interpolated string `"{strategy}:{text}"` and a byte array from `Encoding.UTF8.GetBytes`. The new implementation:

- Rents a character buffer from `ArrayPool<char>.Shared` and concatenates `strategy + ':' + text` into it.
- Rents a byte buffer from `ArrayPool<byte>.Shared` and encodes UTF-8 directly into it (`Encoding.UTF8.GetBytes(span, span)`).
- Computes SHA-256 into a stack buffer `stackalloc byte[SHA256.HashSizeInBytes]` and formats hex.
- Guarantees that the rented buffers are returned in `finally`.

Result: on the “warm” compression-cache path, intermediate allocations of large strings/arrays are eliminated, which reduces GC pressure for large volumes of debate context.

3.4. ValueTask in the Operator role

The Operator resource-disposal method is implemented as `public async ValueTask DisposeAsync()` (`src/Delibera.Core/Council/Operator.cs`) — the standard .NET 10 `IAsyncDisposable` pattern, which avoids a `Task` allocation on the MCP-client disposal path. The new example (`OperatorMcpToolsExample`) uses `await using`, correctly exercising this path.

Optimization summary

Technique	Where applied	Effect
<code>ReadOnlySpan<T> + SIMD</code>	<code>SemanticCompressor.CosineSimilarity</code>	8-16× processing width, no copies
<code>ReadOnlySpan<char></code>	<code>TokenCounter</code> (word/token counting)	No substring allocations
<code>ArrayPool<T> + stackalloc</code>	<code>CompressionCache.ComputeKey</code>	−2 heap allocations per call, less GC pressure
<code>ValueTask</code>	<code>Operator.DisposeAsync</code>	Disposal without a <code>Task</code> allocation

4. Operator + MCP Tools Examples

A new example `OperatorMcpToolsExample` was added (`src/Delibera.ConsoleApp/Examples/OperatorMcpToolsExample.cs`), demonstrating the **Operator** role working with real MCP servers.

Connected MCP servers

Server	Transport	Launch command	What it provides
 browser	stdio	<code>npx -y @playwright/mcp@latest --headless</code>	Site navigation, page reading, clicks, screenshots
 marp	stdio	<code>npx -y @marp-team/marp-cli --server <dir></code>	Generating presentations (HTML/PDF/PPTX) from Markdown

The example also shows (commented out) an HTTP/SSE variant via `McpServerConfig.Http(...)` for remote MCP servers with authorization headers.

What the example demonstrates

Section A — direct Operator call (without a council):

- Manual creation of MCP clients (`McpClientAdapter`) and an `Operator` object.
- `await using` for correct disposal via `ValueTask DisposeAsync` .
- `InitializeAsync()` — connecting to the servers and discovering tools.
- Two tasks via `ExecuteTaskAsync` :
 1. **Browser:** open `https://modelcontextprotocol.io` and briefly summarize what MCP is.
 2. **Marp:** generate a 3-slide presentation and save it as `deck.html` .

Section B — Operator inside a council (delegation via the `[[OPERATOR: ...]]` marker):

- Building a council via `CouncilBuilder` with participants, a chairman (`Chairman`), and an Operator (the convenient `WithOperator(modelName, provider, servers)` overload).
- Subscribing to `OnRoundCompleted` to print Operator activity per round.
- Saving the debate Markdown report and the generated presentations.

Running the example

```
# from the project directory
dotnet run --project src/Delibera.ConsoleApp -- --operator-mcp
```

Prerequisites

- **Node.js + npx** — required to launch the npx MCP servers (`@playwright/mcp` , `@marp-team/marp-cli`).
- For the browser, on first run Playwright will download the engines: `npx playwright install` .
- **Ollama** running locally (`ollama serve`) with the models `llama3.2` , `qwen2.5` (the Operator model can be replaced with any “cheap” one).

The example is resilient to a missing environment: on error it prints clear hints, and the process does not crash.

5. Build and Run

```
# Build both projects
dotnet build

# Build in Release
dotnet build -c Release

# Run the console application
dotnet run --project src/Delibera.ConsoleApp

# Available demos (flags):
dotnet run --project src/Delibera.ConsoleApp -- --operator           # basic Operator ex-
ample                                                                ample
dotnet run --project src/Delibera.ConsoleApp -- --operator-mcp      #  browser + Marp
dotnet run --project src/Delibera.ConsoleApp -- --compression       # context compression
dotnet run --project src/Delibera.ConsoleApp -- --rag               # RAG
dotnet run --project src/Delibera.ConsoleApp -- --di                # Dependency Injec-
tion                                                                tion
```

Build result:

```
Build succeeded.
    0 Warning(s)
    0 Error(s)
```

Note about localhost. The examples use the address `http://localhost:11434` (Ollama) and local paths for presentation output. This is the localhost of the machine where the application runs (the Abacus AI Agent environment), not your computer. To run it locally, download the files via the “Files” icon, go into the downloaded folder, deploy the application on your own system, and run it.

6. Summary of Changed Files

File	Change
src/Delibera.Core/Delibera.Core.csproj	LangVersion=preview , SourceLink → 10.0.300
src/Delibera.ConsoleApp/Delibera.ConsoleApp.csproj	LangVersion=preview
src/Delibera.Core/Compression/SemanticCompressor.cs	SIMD vectorization of CosineSimilarity + ReadOnlySpan<float>
src/Delibera.Core/Compression/TokenCounter.cs	ReadOnlySpan<char> overload, span-based word counting
src/Delibera.Core/Compression/CompressionCache.cs	ArrayPool<T> + stackalloc in ComputeKey
src/Delibera.ConsoleApp/Examples/OperatorMcpToolsExample.cs	 Operator example with browser and Marp
src/Delibera.ConsoleApp/Program.cs	--operator-mcp flag, fixed nullable warning

Document prepared as part of the Delibera upgrade to .NET 10 / C# 15.